

TeXMol User Guide

Vinay K Siddavanahalli

Contents

Contents	2
Preface	5
1 Introduction	7
1.1 Functions	8
1.2 Installation	8
1.2.1 Step by step installation guide	9
1.2.2 Common installation problems	9
1.3 The main user interface	10
2 Opening and rendering files	13
2.1 Loading a molecule description file	13
2.1.1 The static hierarchy for biomolecules	13
2.2 Loading volume files	16
2.3 Loading surface files	16
3 Computations	19
3.1 Isosurfaces	19
3.1.1 Rendering isosurfaces	19
3.2 Curvatures	19
3.3 Electron density	20
3.4 Hydrophobicity	23
3.5 Contour spectrum	23
4 Scripting and animations	27
4.1 Batch-mode calculations	27
4.1.1 Electron density and hydrophobicity	27
4.1.2 Curvatures	28
4.2 Scripting	29
4.3 Animations	30
4.3.1 Creating a trajectory	30
4.3.2 Saving animations	30
Acknowledgements	33

Bibliography**35**

TexMol

This is the user documentation for *TeXMol* v1.0. This is to help both new users familiarize themselves with the software, and to provide a comprehensive list of functions to everyone. There is a separate programmer documentation available for those interested in extending the functionality of *TeXMol* , which is an open source software. *TeXMol* was developed at the Center for Computational Visualization under Dr. Chandrajit Bajaj, at the University of Texas at Austin.

Chapter 1

Introduction

Computational visualization of large molecules – particularly for protein and RNA structures – has gained tremendous importance as a cutting-edge tool for biological research. Previous work has focused on efficient rendering for single-component, static molecules, which is becoming increasingly restricted in light of the increasing demand for more complex, dynamic visualization and representation. We present *TeXMol* (short for Texture Molecular Viewer), an interactive molecular exploration package created in response to the increasingly demanding visualization needs of the biology community.

To efficiently visualize dynamic and flexible structures, *TeXMol* uses a molecular specification file to construct the Flexible Chain Complex (FCC), a robust, dynamic data structure that serves as *TeXMol*’s internal representation for molecular structures. The FCC models the flexible joints of a molecule and contains a biochemically-based hierarchy for level-of-detail optimizations.

Besides high visualizer functionality, *TeXMol* delivers rapid, accurate rendering via various novel applications of texture-based rendering techniques for structural and for volumetric representations. For the field of structural representation (e.g. CPK (of union of balls, where each atom is represented by a sphere with the radius equal to the van der Waals radius.), ball-and-stick model), recent advances in programmable graphics hardware have opened the door for texture-based rendering – also known as imposter rendering – that greatly reduces geometric complexity while preserving, and in some cases improving the visual fidelity of the final image. *TeXMol*’s level-of-detail hierarchy allows for static and dynamic multiresolution, which reduces the visual clutter that often accompanies atom-level visualization while still maintaining biochemical structural information, such as residue-level grouping. Combined with the level-of-detail hierarchy, texture-based rendering allows *TeXMol* to render large and previously intractable molecules.

TeXMol also supports efficient volumetric visualization via texture-based techniques. By combining rendering modes, the visualizer can either map volumetric data onto the structural model of the molecule or it can juxtapose multiple volume sets and structure models concurrently. In both cases, the resulting

visualization ties molecular structure to molecular function in an elucidating manner.

1.1 Functions

TeXMol is both a visualization and computational toolkit for large biomolecules. Some of the main features include

- Open source software for rendering large molecule data sets. It has been tested on molecules with more than 3million atoms.
- Written in C++, with a QT front end. The same code should work on multiple platforms, including Windows and Linux.
- Reads in the well known PDB and PQR formats.
- Produces high quality images using High-end graphics cards functionalities.
- Volume rendering and calculation of electron density, hydrophobicity functions.
- Wireframe and smooth shaded isosurfaces.
- Computes metrics including curvatures, surface areas and volumes.
- Provides programmers with a simple and yet powerful hierarchical data structure of the molecules, with various torsion angles calculated.
- Multiple views and multiple data set rendering.

1.2 Installation

Please install *TeXMol* and perform the functions described in the rest of the tutorial to learn more about the software. *TeXMol* is open source and should be available for download from CCV's software download page. It has been tested under both windows and linux. You will need the following

- A windows or linux operating system.
- QT and a C++ compiler.
- A high end graphics card, like GeForce fx or better.
- An input PDB file, which can be downloaded off the PDB database.

Once you have downloaded the source, you can compile it using either the QT's Makefiles (the .pro files) or the project workspace files (for windows).

1.2.1 Step by step installation guide

There are some subtle problems which could come up when compiling and linking *TeXMol*. Here are some solutions.

- Check to ensure that you have a good graphics card. We have currently tested our programs on NVidia's cards, GeForce fx and beyond.
- Upgrade your graphics card drivers and test example Cg programs they usually provide. There should be examples on NVidia's website, which can help you ensure that it is set up well.
- Make sure that the latest Cg compiler is installed.
- Define the environment variables CG_BIN_PATH, CG_LIB_PATH and CG_INC_PATH if it was not already done.
- Download and install QT. There is a free version for Linux.
- Make sure that the variables QTDIR and QTLIB are defined.
- If the contour library is being used, the user, unfortunately needs to define whether they are using a big endian or little endian machine. Hence, you may or may not need to define the variable LITTLE_ENDIAN. Other libraries do not need this and can figure out the endianness in the code.

1.2.2 Common installation problems

- Compilation problems
 - *QT and X11 incompatibility*

You could get into problems due to QT and X11 defining things arbitrarily. One way to get around this is to change some headers or redefine the error causing terms.
 - *Gl extensions*

Download the latest glxext.h from SGI's website. DELETE other vendors glxext.h. Before installing *TeXMol*, check to see if Gl and Cg demos are working.
 - *file not found*

QT's internal compilers may not have created the files needed by a library in a subfolder of *TeXMol*. Try compiling a different subfolder or the main folder itself and try again.
- Rendering issues
 - *Doesn't render*

Cg files provided are linked in at run time. So you will need to keep them in the same folder as *TeXMol*'s executable.

1.3 The main user interface

In figure 1.2.2, we show a volume rendering and isosurface of a molecule. The different regions in the user interface are

- Molecule browser. The list of molecules are shown here. The name is displayed, and if it was left blank on opening the file, the full filename is shown. The user can select files to either manipulate it using the properties window or to delete it.
- The rendering area. This is either in perspective or orthographic mode, runs using OpenGL and can be either a single or multiple views.
- Properties area. Each type of file (volume, molecule or surface) has its own unique properties widget which is displayed here. On selecting a data set from the browser, its corresponding properties are displayed here.

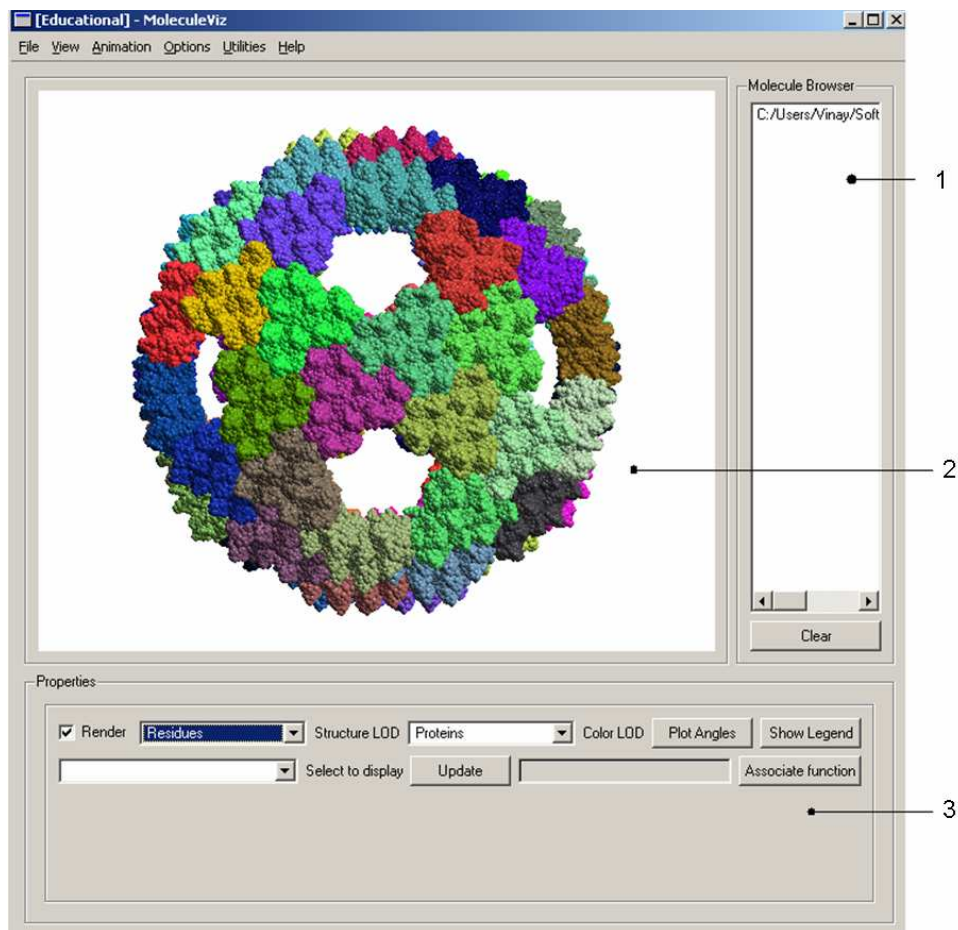


Figure 1.1: The main user interface of *TeX Mol*. In this figure, we mark the three main regions users should get familiar with: 1. *Molecule browser* where data sets names are displayed 2. *Rendering area*, where data is rendered and 3. The *Properties widget* where the properties of the selected data set is shown.

Chapter 2

Opening and rendering files

TexMol reads molecule-specific PDB, PQR files (including simpler XYZ, XYZR, PTS and a custom GOA formats), volume files (RAWIV, RAWV, DX, MRC), surface files (RAW, RAWC, RAWN, RAWNC, OBJ, C2C) and a custom NURBS file. By default, the data set is neither centered to the view, nor rendered to screen. The user needs to specifically render the data set after loading it.



Warning: If you do not have a good high end graphics card, or if it is not set up properly to display using Cg, you will not see any output on the screen. We are in the process of moving to OpenGL Shading Language.

2.1 Loading a molecule description file

We support multiple molecular files. The most commonly used file formats are PDB [2] and PQR [3]. We also have simple file formats to describe general point sets: XYZ (list of centers on successive lines), XYZR (same, includes radius next to center) and PTS (similar to XYZ, but first line contains number of centers). The properties box for molecular data sets is shown in figure 1.2.2. There are two LODs as shown in the figure, the color LOD and the object LOD.

2.1.1 The static hierarchy for biomolecules

We use the biochemical hierarchy in our visualizations. For atoms, we use the CPK model, with van der Waals radii. The colors and radius used are described in the table in *elementInformation.h*. Atoms are grouped into residues, which are approximated by spheres in the structure LOD. Secondary structures are rendered with imposters like cylinders and helices. Chains are rendered as a ball-and-stick model.

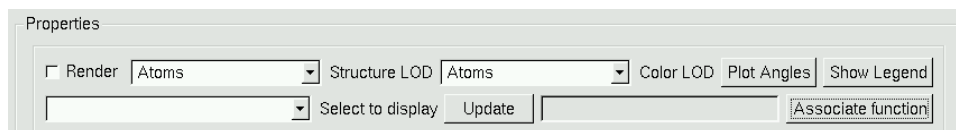


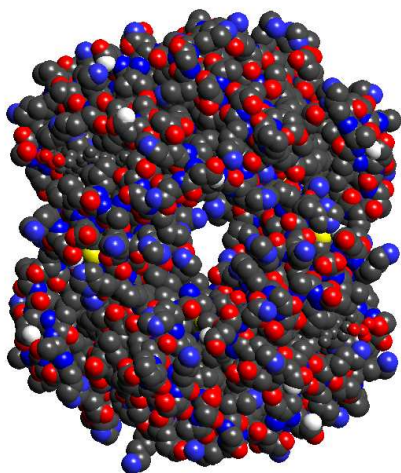
Figure 2.1: The *Properties widget* for molecule data sets allows users to pick their choice of color and structure level of detail to render. Other functions include associating a volumetric function with the surfaces color.

- Atoms. The atom sequence is the lowest, or finest level in the hierarchy to obtain the primary structure of the molecule.
- Residues. Atoms are grouped into their respective residues.
- Secondary structures. Residues are grouped in to either helices or sheets, or into dummy NULL structures.
- Chains. The secondary structures are collected in to chains. They form the tertiary structure.
- Molecule. The highest or coarsest level in the static hierarchy is the molecule itself.

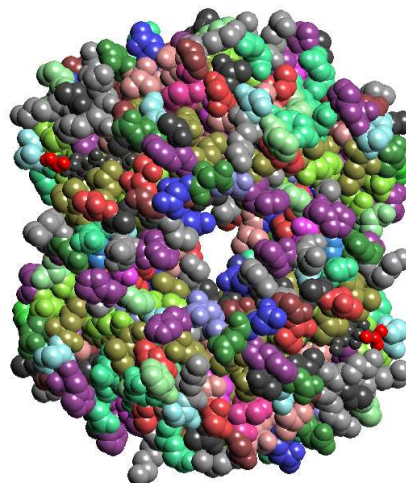
After loading the molecule file, one can select a combination of color and structure LOD to obtain various visualizations. To open a PDB or PQR file, follow the steps shown in table 2.1. A simple well known molecule to start with would be hemoglobin (1A00.pdb), as it has multiple chains and helices. Some interesting combinations of visualizations would be as shown in figure 2.2. Table 2.2 explains how one can visualize the helices in the loaded molecule.

Table 2.1: Loading a molecule

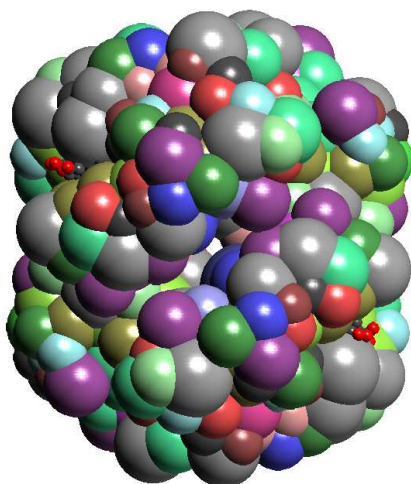
- Select File - Open from menu bar.
- Choose a file name by pressing the File button and select a PDB or PQR file.
- If you know that you are choosing a isosahedral virus, check the isVirus check box.
- Press OK to load the file, or Cancel to cancel this action.



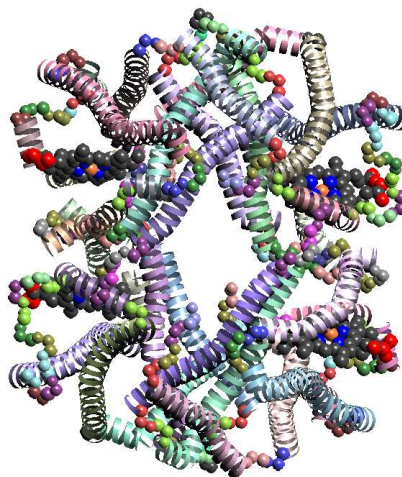
(a) Atoms colored with element colors



(b) Atoms colored with their respective residues colors



(c) A coarser model with residues rendered as spheres



(d) Helices forming the secondary structures in the molecule

Figure 2.2: Molecules can be rendered at different color and structure level of details. Here we show the hemoglobin molecule (1A00.pdb) rendered in 4 different styles.

Table 2.2: Visualizing helices in a molecule

- If you have not loaded a molecule, do so by following the steps outlined in Table 2.1.
- Set the Structure LOD to Secondary structures and the Color LOD to any item.
- Render the molecule by checking the Render check box.
- If the molecule is not visible in the current view point, you may need to zoom out and translate to bring it in to view.

2.2 Loading volume files

We currently support four volume files including the scalar RAWIV, MRC and DX formats, and vector valued RAWV formats. They are both binary and ascii files and support multiple data types and sizes. For a complete description, see the file format descriptions on the CCV web site. To load volume files, follow the same steps as outlined in table 2.1, but select a volume file instead of a molecule file.

To render volume files and isosurfaces, you need to get familiar with a widget called a color table, which is user interface for reading a transfer map. A detailed description for the color table can be found in the volume rover software user guide.

2.3 Loading surface files



Warning: Surface files currently supported can be compressed or not. The uncompressed files can take up to a few minutes to load for very large meshes.

Like volume files, surface files do not need to deal with molecules. *TeXMol* currently supports uncompressed (RAW, RAWC, RAWN, RAWNC and OBJ) and compressed (c2c) surface files. To load any of these file formats, table 2.1 can be followed with the right input file types. Both wireframe and surface rendering modes are supported. The user can also view both together. A visualization of the Mache molecule, showing the gorge is shown in figure 2.1.1. The OBJ file format is from Alias Wavefront.

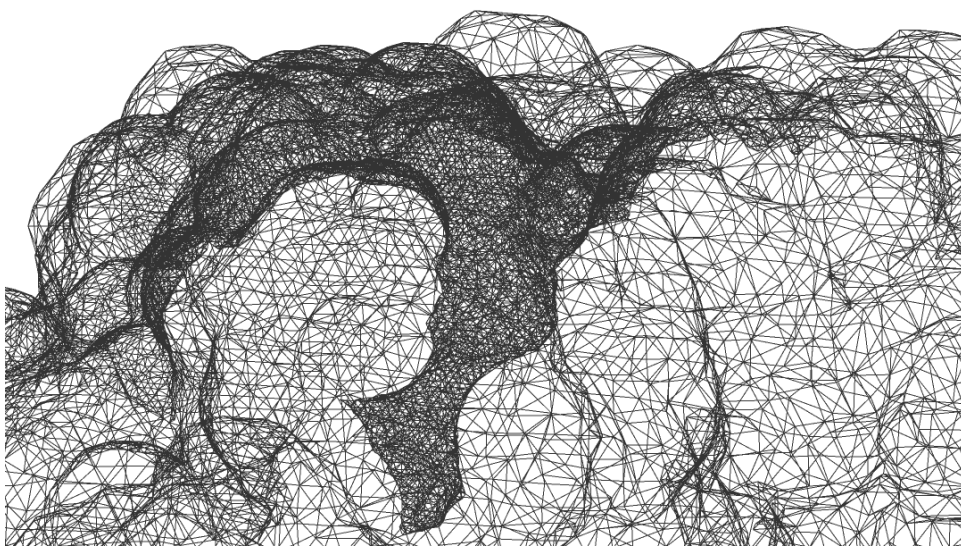


Figure 2.3: Wireframe rendering of a mesh, showing a gorge type feature in the surface of the MACHE molecule.

Chapter 3

Computations

Apart from visualization, *TeXMol* also allows users to compute functions of the molecule including surfaces and surface functions like isosurfaces, curvatures, and the contour spectrum and volume functions like electron density and hydrophobicity.

3.1 Isosurfaces

Given a volume $f(x, y, z)$, an isosurface with isovalue c is defined as $f(x, y, z) = c$. The extraction of isosurfaces from volume files is known as isocontouring. There are two packages for isocontouring within *TeXMol*. One is the *marching cubes* algorithm implemented in the *Contouring* library and the other is the *seed set* algorithm implemented in the *contourlib* library. To create an isosurface, the user needs to load in a volume and extract it at a given isovalue as described in table 3.1.



Warning: Isosurfacing of large volume files could take up to a minute.

3.1.1 Rendering isosurfaces

Isosurfaces are rendered by default on creation and cannot be hidden. We also currently use smooth shading for rendering the meshes. The color of the mesh at any vertex is given by the color of the volume at that point. The color of the entire mesh can be changed by right clicking on the isocontour bar and selecting edit.

3.2 Curvatures

Using a functional definition for the electron density representation of a molecule, the curvatures at any point can be estimated. Analytical definitions

Table 3.1: Isosurfacing

- Load a volume if it is not already done and select it in the molecule browser to bring up its property widget.
- In the color table, right click at any point and add an isocontour.
- This could take a few seconds to a minute or so.
- The isovalue can be modified by dragging the isosurface bar in the color table.
- The isosurface can be deleted by deleting its isosurface bar.

of the functions yield analytical equations for deriving the curvatures. *TeXMol* estimates two curvatures, the Mean curvature H and the Gaussian curvature K . If k_{min} and k_{max} are the minimum and maximum curvatures at a point, then

$$H = \frac{1}{2}(k_{min} + k_{max}), K = k_{min} \times k_{max} \quad (3.1)$$

These are calculated from a potential function ϕ as follows

$$H = \frac{C(f_x^2(f_{yy} + f_{zz})) - 2C(f_x f_y f_{xy})}{2(C f_x^2)}, K = \frac{2C(f_x f_y (f_{xz} f_{yz} - f_{xy} f_{zz}))}{(C(f_x^2))^2} \quad (3.2)$$

C denotes cyclic summation over x, y, z , subscripts denote partial differentiation with respect to those variables.

There are two ways in which most functions can be calculated in *TeXMol*. First, the user can do it through the graphical user interface. Second, it can be done in batch mode. In table 3.2, we describe how to calculate the mean and gaussian curvatures from the graphical user interface.

3.3 Electron density

The electron density has been used as a description of molecular shape. One commonly used model of sum of radial functions has been used in *TeXMol*. This field can be described at any point x, y, z in a volume as a scalar value given by

$$\phi_{dens} = \sum_{i=1}^M e^{\beta_i} \quad (3.3)$$

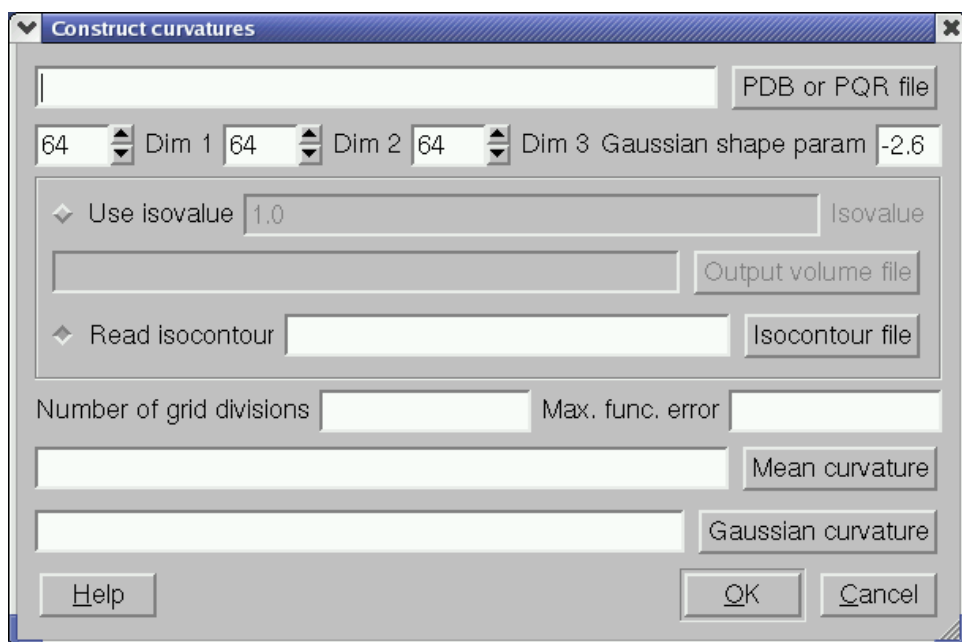


Figure 3.1: This curvature widget can be opened by selecting Utilities - Construct curvatures from the main menu bar.

Table 3.2: Estimation of curvatures from GUI

- Consider figure 3.1.1 for this example.
- Load a molecular file to estimate curvatures for.
- Since we need an electron density function definition, select values for the volume dimensions and the gaussian blobbyness parameter.
- If you know the points at which to estimate the curvature, you can load the mesh file, or ask *TeXMol* to perform isocontouring at some selected isovalue.
- The number of grid divisions and the function error allowed are tradeoffs to performance.
- Three files, the mean and gaussian curvature, and the curvature values themselves are written out to the selected files.

and

$$\beta_i = \text{blobby} * \frac{(x - xc_i)^2 + (y - yc_i)^2 + (z - zc_i)^2}{vr_i^2} - \text{blobby} \quad (3.4)$$

M is the total number of atoms,
 xc_i, yc_i, zc_i is the center of the i^{th} atom,
 vr_i is the van der Waal radius of atom i and
 blobby is a parameter which controls the shape of the gaussian.

Using the graphical user interface, the users can compute the electron density of a molecule as shown in table 3.3. See [4] for a fast summation algorithm. There are four types of electron density volumes we compute with respect to visualization.

1. Scalar valued electron density.
2. Vector valued electron density, where there are 4 tuples at each point. RGB at a point contains a color value, and A contains the density itself. This can be computed in three ways. See figure 3.2 for an example.
 - (a) Color according to a structure like atoms, residues etc.
 - (b) Color using the specifications in a color map file.
 - (c) Use depth coloring to bring out surface features. This feature currently takes as input a volume file (scalar or volume). Load a

volume file and select Utilities, Construct depth colored volume from the menu bar.

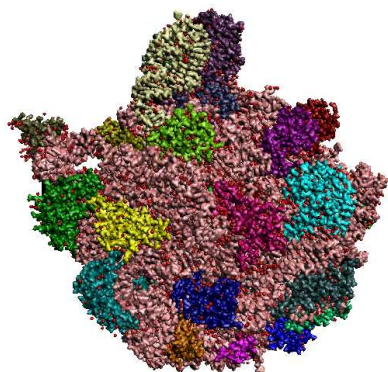
Table 3.3: Electron density computation

- Load a molecule file if it is not already loaded.
- Select the molecule from the molecule browser.
- Select Utilities and Construct volume from the menu bar.
- Select Electron density as the function needed to be computed and give the dimensions and blobbyness parameter.
- If you need a output file, then enter a output file name, or just leave it blank.
- RawV volumes or vector valued volumes can also be constructed by choosing it in the check box.
- If rawV was chosen, then also choose a color LOD for the density.
- The generated volume can be loaded in to memory or not by checking Load generated volume.

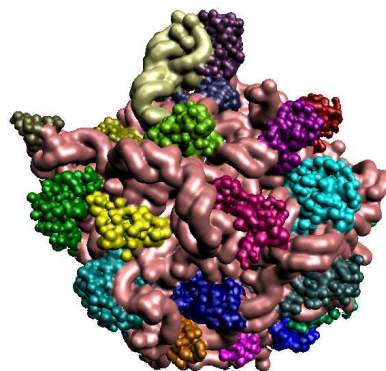
3.4 Hydrophobicity

Hydrophobicity maps can be created in a similar fashion as rawiv electron density files. Follow the procedure outlined in table 3.3, but choose hydrophobicity instead of electron density. You probably cannot create anything meaningful by choosing hydrophobicity and color mapped volumes together. The hydrophobicity is given per atom as shown in the table in file *elementInformation.h*. The hydrophobicity of the dengue virus capsid protein (1r6r.pdb) is shown in figure 3.1.1.

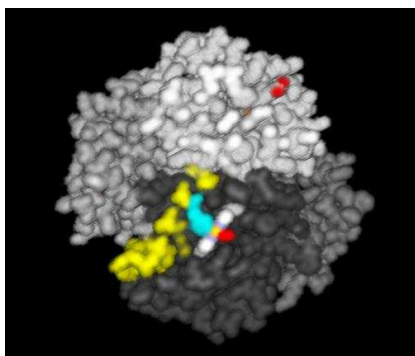
3.5 Contour spectrum



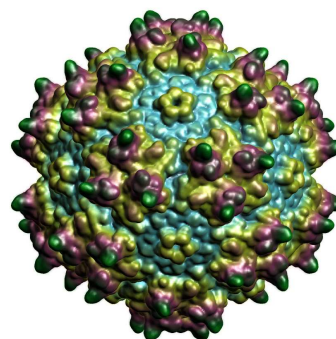
(a) Volume with colors assigned to secondary structures, blurred at the atomic level



(b) Volume with colors assigned to secondary structures, blurred at the residue level



(c) In order to show specific atoms like the iron in the heme structure of the hemoglobin, we can use a color map file.



(d) A virus (1lp3.pdb) rendered as an iso-surface of a depth colored volume showing its surface features

Figure 3.2: Vector valued volumes constructed in different ways to bring out different features.

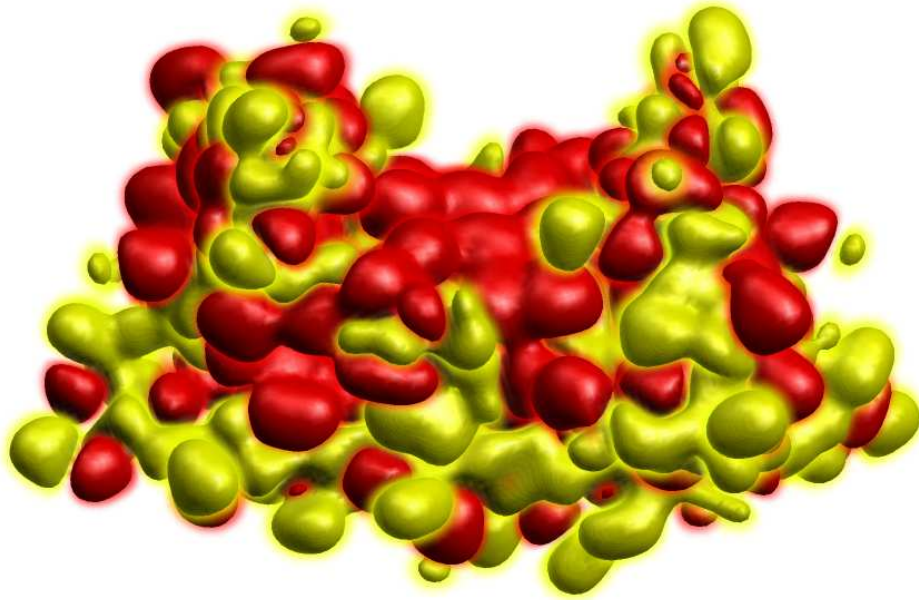


Figure 3.3: Hydrophobicity volume rendering of the dengue virus capsid protein (1r6r.pdb). The hydrophobic regions are in yellow and the hydrophilic regions are in red.

The contour spectrum is used for obtaining quantitative information about the volume files. The documentation for the contour spectrum can be found in the user manual for CCV's volume rover.

Chapter 4

Scripting and animations

The computations defined in chapter 2.1.1 can also be performed in batch mode. This is useful when a large number of files need to be manipulated at once. Two other powerful features of *TeXMol* include

- Scripting.
- Animation.

In this chapter, we will go through simple examples for each of the above three features.

4.1 Batch-mode calculations

Electron density, hydrophobicity and curvatures can be estimated from the command line. The commands are as follows.

4.1.1 Electron density and hydrophobicity

The command line for creating an electron density or hydrophobicity map is:

```
TeXMol -blur [input molecule file name] [output volume file name] [dim1]
[dim2] [dim3] [density type] [colored volume] [gaussian blobbiness] [color] [colormap file name]
[gap]
```

The different parameters are:

[input molecule file name] The full path and name of the input molecule file.

[output volume file name] The full path and name of the output rawiv or rawv file.

[dim1, dim2, dim3] Dimensions of the output volume.

[density type] Enter 0 for electron density and 1 for hydrophobicity.

[colored volume] *true* if you want a rawv volume.

gaussian blobbiness We recommend a value of -2.3 to conform with previous research and bigger values, say -0.1, to get smoother shapes representing the volume at much lower resolutions.

color If the colored volume parameter was true, then we can either specify either color by structure or provide a color map file. If we do not provide a color map file, then this parameter is relevant. 0 stands for atom coloring, 1 for residue, 2 for secondary structure and 3 for chain coloring.

colormap file name The molecule can be colored according to some user defined colors, to perhaps highlight some structure which cannot be done in the more general method previous described. The file format for the *colormap file name* is given in the file format description page from CCV's website.

gap This should be used sparingly, to provide a gap of empty space around the requested dimensions. It is best left as 0.

4.1.2 Curvatures

There are two commands for generating curvatures. One is from a volume file with a given isovalue, where *TeXMol* extracts the mesh where we need to obtain the curvature, and second, where the user inputs the mesh directly. The two commands in order are:

```
TeXMol -setcurvature 1 input molecule file name output volume file name
output mean curvature file name output gaussian curvature file name dim1
dim2 dim3 gaussian blobbiness isovalue number of grid subdivisions maximum function error
```

```
TeXMol -setcurvature 0 input molecule file name input surface file name
output mean curvature file name output gaussian curvature file name dim1
dim2 dim3 gaussian blobbiness number of grid subdivisions maximum function error
```

The various parameters are described below.

0 and **1** differentiate between the two calls.

input molecule file name The full path and name of the input molecule file.

output volume file name The full path and name of the output rawiv or rawv file.

output mean curvature file name The full path and name of the output mean curvature file.

output gaussian curvature file name The full path and name of the output gaussian curvature file.

dim1, dim2, dim3 Dimensions of the output volume.

gaussian blobbiness We recommend a value of -2.3 to conform with previous research and bigger values, say -0.1, to get smoother shapes representing the volume at much lower resolutions.

number of grid subdivisions The higher this value, the more the precomputation and memory requirement. There is a cut off value around 10 to 20 where you get optimum speed for a given machine.

maximum function error The maximum allowable function error while calculating the curvatures.



Warning: While creating curvatures, if we overestimate the value of grid spacing, we could end up with very high memory usages and may have to kill the program.

4.2 Scripting

TeXMol currently offers a limited but powerful set of functions as part of its scripting module. This module is available once *TeXMol* is run in user interface mode. The parser command window can be opened from Utilities - script from the main menu bar. Here users can enter commands, select one or more and execute them. Some of the commands available are listed below.

```
bool blur(int argc, char* argv[]);
bool setCurvature(int argc, char* argv[]);
bool outGridPositions(int argc, char* argv[]);
bool classifyPoints(int argc, char* argv[]);
bool growOut(int argc, char* argv[]);
bool getSurface(int argc, char* argv[]);
bool evolve(int argc, char* argv[]);
bool writePDB(int argc, char* argv[]);
bool writeGOA(int argc, char* argv[]);
bool depthColor(int argc, char* argv[]);
bool printPDBInformation(int argc, char* argv[]);
bool getMaxDistanceFromPoint(int argc, char* argv[]);
bool addNewDataSet(int argc, char* argv[], MoleculeVizMainWindow *mWin-
dow );
bool splitView(int argc, char* argv[], MoleculeVizMainWindow *mWindow
);
bool deleteData(int argc, char* argv[], MoleculeVizMainWindow *mWindow
);
bool deletePrevData(int argc, char* argv[], MoleculeVizMainWindow *mWin-
dow );
bool deleteAllData(int argc, char* argv[], MoleculeVizMainWindow *mWin-
dow );
bool setVisible(int argc, char* argv[], MoleculeVizMainWindow *mWindow
);
bool setVisiblePrev(int argc, char* argv[], MoleculeVizMainWindow *mWin-
dow );
```

```

    bool saveImage(int argc, char* argv[], MoleculeVizMainWindow *mWindow
);
    bool setGridVisible(int argc, char* argv[], MoleculeVizMainWindow *mWin-
dow );

```

4.3 Animations

This module is mainly useful for making movies. Animations can be saved as a sequence of images in *TeXMol*. We allow the user to load a data set, render it and transform the view point, and save trajectories and play it back. There are actually two steps in the animation process.

1. Create and save a trajectory.
2. Save the animation based on a trajectory.

4.3.1 Creating a trajectory

Data sets can be quite large, making them hard to interact with. Hence, it is sometimes useful to save a trajectory using a low resolution data set and later use it to render a large data set in high resolution. In order to create the trajectory it is useful to have a low resolution data set which the user can comfortably rotate, zoom and translate interactively. To save a trajectory, follow the procedure outlined in table 4.1

4.3.2 Saving animations

Before you can save a sequence of images, you need to create a trajectory as described in section 4.3.1.



Warning: Disable the screensaver as saving images in high resolution can take a long time. We also interpolate between mouse movements using linear interpolation to get better smoothness, causing the movie to be longer than expected.

Table 4.1: Creating and saving a trajectory

- Load and display the data set or set of data sets you want to create a movie of.
- Use the mouse to move them into the correct starting viewing position.
- Click Animation, Start recording from the menu bar.
- Enter and save the file name of the trajectory file. It does not require any specific file extension.
- On pressing OK, the recording is on. Any mouse movement in the rendering area is saved in to the file, along with the time.
- Once you are done with transforming the data, click Animation, Stop recording in the menu bar.
- You can see if the trajectory was satisfactory by playing it back by clicking on Animation, Playback animation from the menu bar.

Table 4.2: Saving an animation

- Load the high resolution data sets to save.
- keep the screen resolution and window size as high as required.
- Open the trajectory file by selecting Animation, Record animation from the menu bar.
- *TeXMol* will automatically begin to replay the trajectory and save images.

Acknowledgements

TeXMol was developed by Vinay K Siddavanahalli under Prof Chandrajit Bajaj.
Some of the other contributors include

- Peter Djeu
- Anthony Thane
- John Wiggins
- Xiaoyu Zhang

Bibliography

- [1] Chandrajit Bajaj, Peter Djeu, Vinay Siddavanahalli and Anthony Thane
TeXMol : Interactive Visual Exploration of Large Flexible Multi-component Molecular Complexes IEEE Visualization 2004.
- [2] H.M. Berman, J. Westbrook, Z. Feng, G. Gilliland, T.N. Bhat, H. Weissig, I.N. Shindyalov, P.E. Bourne: The Protein Data Bank. *Nucleic Acids Research*, 28 pp. 235-242 (2000).
- [3] Todd J. Dolinsky, Jens E. Nielsen, J. Andrew McCammon, and Nathan A. Baker: PDB2PQR: an automated pipeline for the setup of PoissonBoltzmann electrostatics calculations. *Nucleic Acids Res.* 2004 July 1; 32
- [4] Chandrajit Bajaj and Vinay Siddavanahalli, Fast Error-bounded Surfaces and Derivatives Computation for Volumetric Particle Data, ICES Report 06-03, January 2006.